

Exercise 1: Rational numbers

In this problem, we will write a class that can represent rational numbers, i.e. fractions p/q .

- (a) Create a class Rational which is initialized by two integers, p and q , the nominator and denominator
- (b) Add a method to print the rational number as p/q (the `__str__` or `__repr__` method is useful).
- (c) We would like to represent $10/20$ by $1/2$ instead, hence write a function that computes the greatest common divisor, and ensure that every rational number is simplified
- (d) Add a method so that we can add two rational numbers with $r1 + r2$, here the `__add__()` method is useful.
- (e) Add a method to subtract two rational numbers. (`__sub__`)
- (f) Add a method to multiply two rational numbers. (`__mul__`)
- (g) Add a method to divide two rational numbers. (`__div__`)
- (h) Add a method that compares whether two rational numbers are equal.
- (i) Add a method to convert the rational number to a floating point (the `__float__()` method maybe handy).

Exercise 2: Sparse and dense vectors

Let's implement class which contain some vector functions. There are two types of vectors, normal or dense vectors, which we can represent using lists. For sparse vectors, where many of the elements are zero, this is inefficient. Instead, we use a dictionary with keys the indices of non-zero values, and then the value corresponding to the key is the value of the vector at that index. Hence, the vector $[1; 2; 4]$ can be stored as a list: `[1, 2, 4]` or as a dictionary `{0:1, 1: 2, 2: 4}`.

we would like to represent sparse and dense vectors as classes, this way we can overload operators such as `+` (`add`) and get sensible output. For example, using `+` on two dense vectors implemented as lists would append the second vector to the first, instead of adding the two together.

Implement sparse and dense vectors. Both classes should have the following capabilities:

- (a) Print vector
- (b) Add two vectors (both if other is dense and sparse)
- (c) Multiply two vectors (both if other is dense and sparse)

Hint: `isinstance()` might be useful.